

Федеральное государственное автономное образовательное учреждение высшего  
образования



**«Московский государственный технический университет  
имени Н.Э. Баумана»**

**(национальный исследовательский университет)**

**(МГТУ им. Н.Э. Баумана)**

---

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ  
КАФЕДРА КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)

## **О т ч е т**

**по лабораторной работе № 4**

**Название лабораторной работы: С#. Обработка массивов, строк и  
подпрограммы**

**Дисциплина: Алгоритмизация и программирование**

Студент гр. ИУ6-23Б \_\_\_\_\_ **С.М. Соболев**  
(Подпись, дата) (И.О. Фамилия)

Преподаватель \_\_\_\_\_ **О.А. Веселовская**  
(Подпись, дата) (И.О. Фамилия)

Москва, 2026

## Часть 1. Массивы.

**Цель работы:** разработать алгоритм преобразования матрицы пересечений дорог в матрицу связей населенных пунктов. Закрепить практические навыки обхода двумерных массивов на языке C++.

**Задание:** есть карта района, включающего несколько населенных пунктов, обозначенных номерами. Пункты попарно связаны дорогами, также помеченными номерами. В квадратной матрице на пересечении строки, номер которой соответствует одной дороге, и столбца, номер которого соответствует другой дороге, указан номер населенного пункта, в котором они встречаются, или 0, если такого пункта нет. Построить матрицу, в которой для каждой пары населенных пунктов указан номер связывающей их дороги. Размерность и элементы матрицы построчно ввести с клавиатуры, результат вывести на экран.

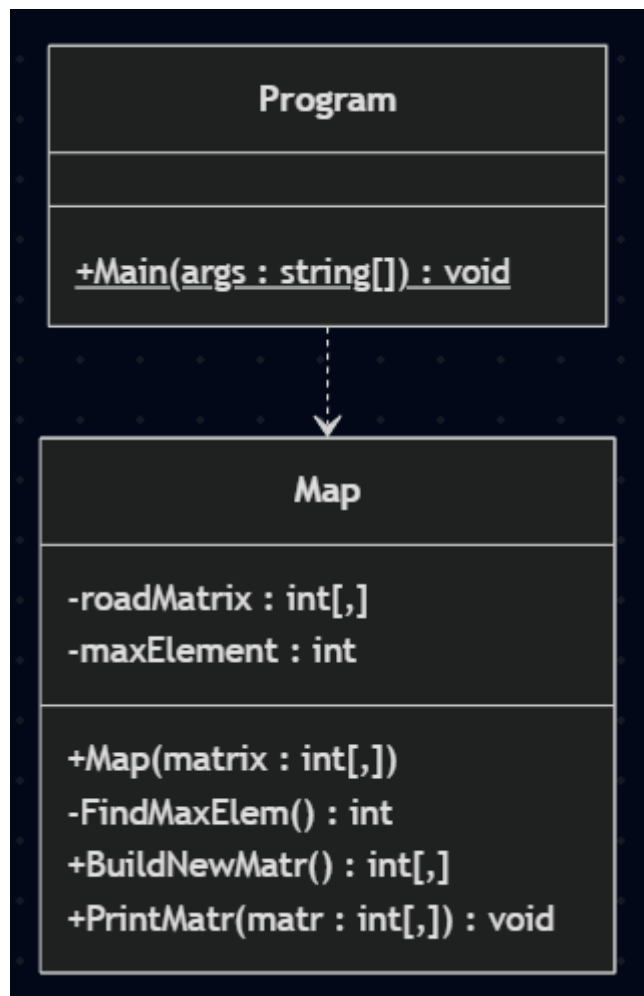


Рисунок 1 – Диаграмма классов

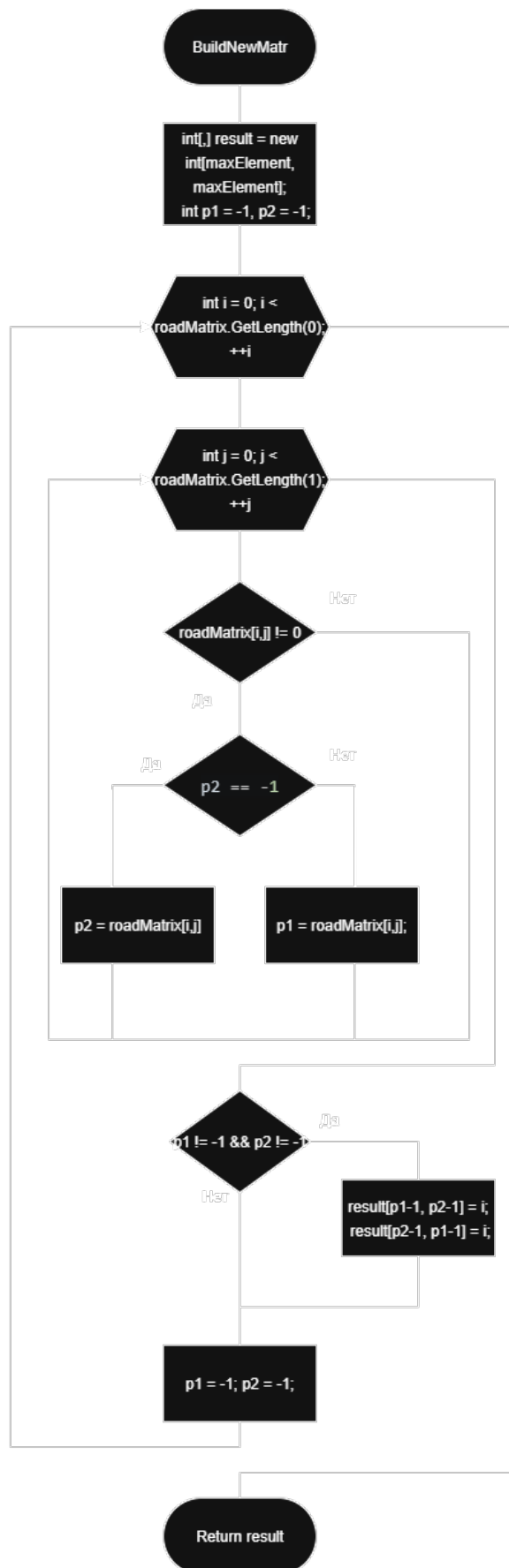


Рисунок 2 – Схема метода BuildNewMatr

```

public int[,] BuildNewMatr()
{
    int[,] result = new int[maxElement, maxElement];
    int p1 = -1, p2 = -1;
    for (int i = 0; i < roadMatrix.GetLength(0); ++i)
    {
        for (int j = 0; j < roadMatrix.GetLength(1); ++j)
        {
            if (roadMatrix[i,j] != 0)
            {
                if (p2 == -1) p2 = roadMatrix[i,j];
                else
                    p1 = roadMatrix[i,j];
            }
        }
        if (p1 != -1 && p2 != -1)
        {
            result[p1-1, p2-1] = i;
            result[p2-1, p1-1] = i;
        }
        p1 = -1; p2 = -1;
    }
    return result;
}

```

Рисунок 3 – Код метода BuildNewMatr

```

Ссылка: 0
static void Main(string[] args)
{
    Console.WriteLine("Введите размер матрицы: ");
    int n = Convert.ToInt32(Console.ReadLine());
    int[,] matrix = new int[n,n];

    Console.WriteLine("Введите матрицу:");
    for (int i = 0; i < n; ++i)
    {
        string[] input = Console.ReadLine().Split(' ');
        for (int j = 0; j < n; ++j)
        {
            matrix[i, j] = Convert.ToInt32(input[j]);
        }
    }
    Map map = new Map(matrix);
    int[,] newMatr = map.BuildNewMatr();
    Console.WriteLine("Новая матрица: ");
    map.PrintMatr(newMatr);
}

```

Рисунок 4 – Код метода Main

```

Ссылка: 1
public Map(int[,] matrix)
{
    roadMatrix = matrix;
    maxElement = FindMaxElem();
}
Ссылка: 1
private int FindMaxElem()
{
    int max = 0;
    for(int i = 0; i < roadMatrix.GetLength(0); ++i)
    {
        for(int j = 0; j < roadMatrix.GetLength(1); ++j)
        {
            if (roadMatrix[i,j] > max) max = roadMatrix[i,j];
        }
    }
    return max;
}

```

Рисунок 5 – Класс Map

```

Введите размер матрицы:
3
Введите матрицу:
1 0 2
3 1 0
0 2 3
Новая матрица:
0 0 1
0 0 2
1 2 0

```

Рисунок 6 – Пример работы программы

Вывод: реализован алгоритм перестроения матрицы. Отработаны механизмы вложенных циклов для поиска и переназначения индексов элементов. Программа корректно выполняет трансформацию связей и вывод результата.

## Часть 2. Строки.

**Цель работы:** создать программу для шифрования и дешифровки текста с помощью циклического числового ключа. Углубить знания алгоритмов посимвольной обработки строк.

**Задание:** дана последовательность строк. Строки содержат слова, разделенные пробелом. Используя цифровой шифр, например 31206, зашифровать каждую строку по следующей методике: 31206 312063 12 063

Пирог сгорел до тла

Ткток фкррло ер есг , то есть, к каждой букве применяют соответствующую цифру для определения смещения этой буквы, с целью получения буквы шифра.

Написать программу, обеспечивающую ввод строк, шифровку и дешифровку. Вывести на экран зашифрованную и подвергнутую дешифровке последовательности строк.

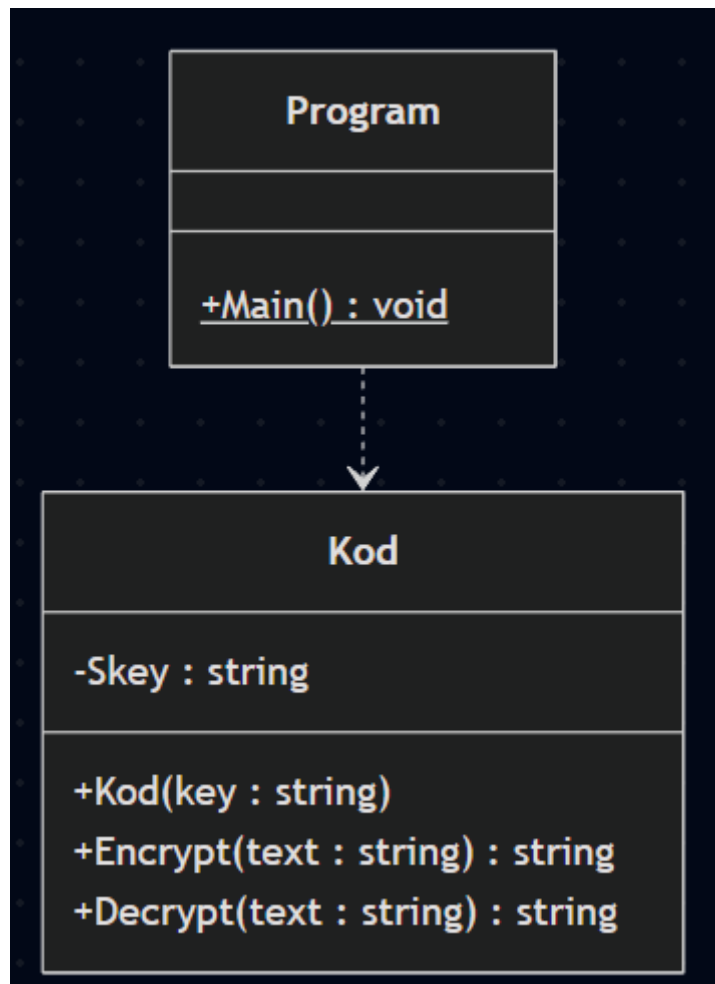


Рисунок 7 – Диаграмма классов

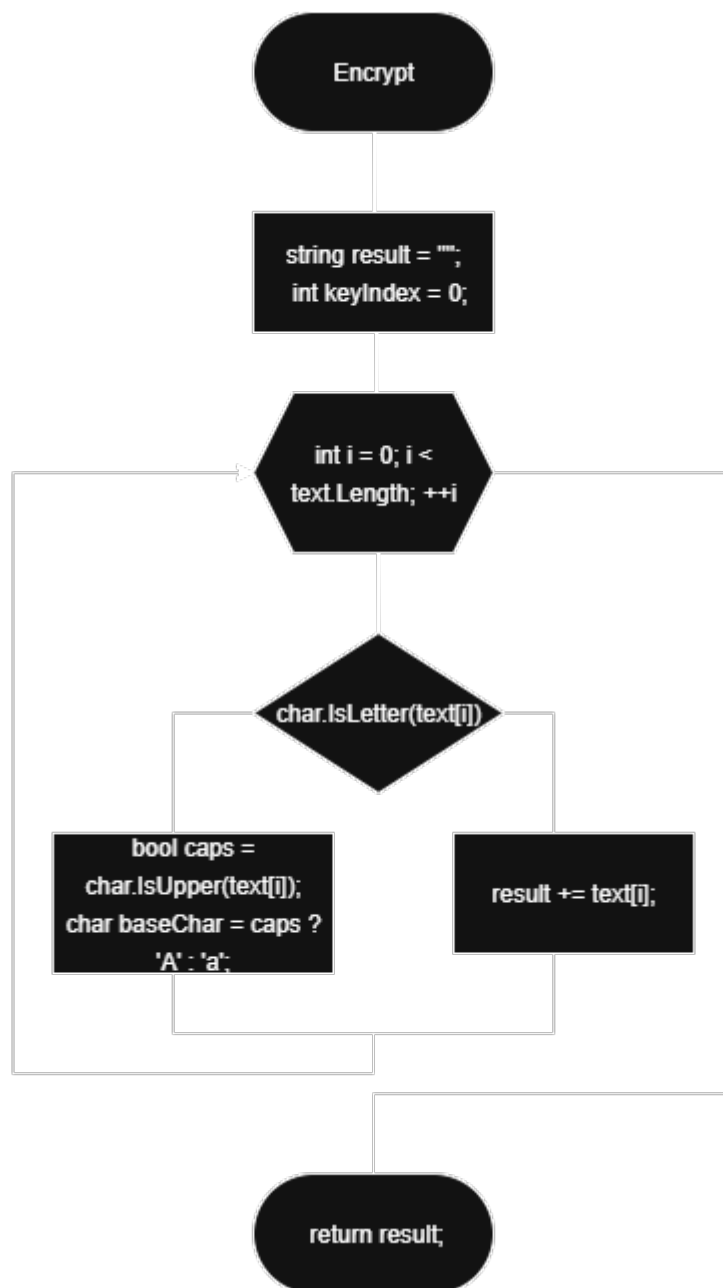


Рисунок 8 – Схема метода Encrypt

```

static void Main()
{
    Console.WriteLine("Введите код: ");
    string k = Console.ReadLine();
    Console.WriteLine("Введите текст: ");
    string text = Console.ReadLine();
    Kod kod = new Kod(k);
    Console.WriteLine(kod.Encrypt(text));
    Console.WriteLine(kod.Decrypt(kod.Encrypt(text)));
}

```

Рисунок 9 – Код метода Main

```

public string Encrypt(string text)
{
    string result = "";
    int keyIndex = 0;
    for (int i = 0; i < text.Length; ++i)
    {
        if (char.IsLetter(text[i]))
        {
            bool caps = char.IsUpper(text[i]);
            char baseChar = caps ? 'A' : 'a';
            result += (char)((((text[i] + Skey[keyIndex]) - '0' - baseChar) % 32) + baseChar);
            keyIndex = (++keyIndex) % Skey.Length;
        }
        else
        {
            result += text[i];
        }
    }
    return result;
}

```

Рисунок 10 – Код метода Encrypt

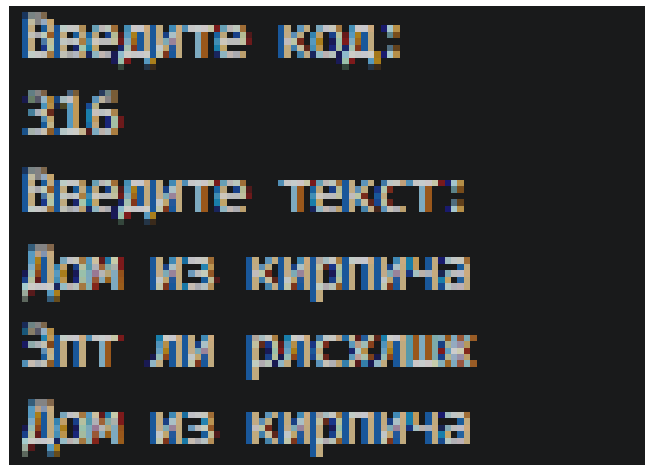
```

public string Decrypt(string text)
{
    string result = "";
    int keyIndex = 0;
    for (int i = 0; i < text.Length; ++i)
    {
        if (char.IsLetter(text[i]))
        {
            bool caps = char.IsUpper(text[i]);
            char baseChar = caps ? 'A' : 'a';
            result += (char)((((text[i] - Skey[keyIndex]) + '0' + baseChar) % 32) + baseChar);
            keyIndex = (++keyIndex) % Skey.Length;
        }
        else
        {
            result += text[i];
        }
    }
}

```

Рисунок 11 – Код метода Decrypt





```
Введите код:  
316  
Введите текст:  
Дом из кирпича  
Эпт ли рлсхлшж  
Дом из кирпича
```

Рисунок 12 – Пример работы программы

Вывод: написан рабочий алгоритм для смещения кодов символов на основе заданного ключа. Успешно реализованы функции прямого и обратного преобразования текста. Получен практический опыт манипуляции со строковыми типами данных в C++.